CS F364: Design & Analysis of Algorithms

Quiz 2 Solution – June 4, 2026

Part (a) — Merging three sorted lists [1 Mark]

To merge three sorted lists of combined size n, we compare the front elements of the (up to) three non-empty lists at each step.

- When all 3 lists are non-empty: 2 comparisons to find the minimum
- When only 2 lists remain: 1 comparison
- The last element requires no comparison

In the worst case, all three lists are exhausted at roughly the same time. Each of the n elements (except the last one) needs $3 - 1 = 2$ comparisons to determine its rank.

$$\boxed{2\,(n-1) \text{ comparisons in the worst case}}$$

Part (b) — Recurrence [1 Mark]

Divide: Split array into 3 parts of size $n/3 \to O(1)$.

Conquer: Recursively sort each part $\to 3\,T(n/3)$.

Combine: Merge 3 sorted lists of total size $n \to 2(n-1) = O(n)$.

$$T(n) = 3\,T(n/3) + O(n), \qquad T(1) = O(1)$$

Master Theorem: $a = 3$, $b = 3$, $f(n) = O(n)$.
$\log_b a = \log_3 3 = 1$, so $f(n) = O(n^{\log_b a}) = O(n^1) \to$ **Case 2**.

$$\boxed{T(n) = \Theta(n \log n)}$$

Part (c) — Comparison with 2-way Merge Sort [3 Marks]

1. Exact running time of 2-way Merge Sort

Merge of 2 sorted lists of size n needs $n - 1$ comparisons.

$$T_2(n) = 2\,T_2(n/2) + (n-1), \qquad T_2(1) = 0$$

Solving:

$$\boxed{T_2(n) = n \log_2 n - n + 1 \;\approx\; n \log_2 n}$$

2. Exact running time of 3-way Merge Sort

Merge of 3 sorted lists of size n needs $2(n-1)$ comparisons.

[T_3(n) = 3,T_3(n/3) + 2(n-1), T_3(1) = 0]

Solving via recursion tree:

$$\begin{aligned} T_3(n) &= \sum_{k=0}^{\log_3 n-1} (2n - 2 \cdot 3^k) \\ &= 2n \log_3 n - (n-1) \\ &= 2n \cdot \frac{\log_2 n}{\log_2 3} - n + 1 \\ &\approx 1.26\, n \log_2 n - n + 1 \end{aligned}$$

$$\boxed{T_3(n) = 2n \log_3 n - n + 1}$$

3. Which is more comparison-efficient?

Compare the leading terms:

Version	Comparisons (approx)
2-way	$n \log_2 n$
3-way	$1.26\, n \log_2 n$

[2-way Merge Sort is more comparison-efficient]

3-way Merge Sort performs about **26% more comparisons** because while the recursion depth decreases ($\log_3 n$ vs $\log_2 n$), the merge step costs $2(n-1)$ instead of $(n-1)$, which more than offsets the shallower recursion.